ShiftSense — Technical Architecture and Data

Version 1.0 · October 2026.

Contents

- 1. Stack
- 2. Architecture
- 3. Planning engine
- 4. Data model
- Entity relationship diagram
- 5. Metric definitions
- 6. AI

1. Stack

Layer	Technology
Language	Python 3.11+ (tested on 3.13)
UI	Streamlit 1.64 (pinned), Plotly charts, HTML components in `ui_kit.py`
Optimiser	Google OR-Tools CP-SAT (time-limited)
Data	pandas, numpy; SQLite (default) or PostgreSQL 16 via `shiftsense/db.py`
Security	PBKDF2-SHA256 passwords; optional OIDC single sign-on (Authlib via Streamlit `st.login`)
Images	Pillow (profile photos: validated, resized, metadata stripped)
Optional	SMTP email copies; Anthropic API for explanations and import help (never decisions)
Deployment	Docker image, Docker Compose (PostgreSQL, app, jobs, Caddy HTTPS)

2. Architecture

One Python process serves both the browser UI and the server logic (Streamlit). The layers are separated by module, not by network:

Layer	Modules	Rule
Screens	`app.py`, `views/*.py`, `ui_common.py`, `ui_kit.py`, `ui_data.py`	Never call domain code to change state directly
Server actions	`shiftsense/services.py`	**Every user action goes through here**: permission, scope, state, audit
Transactions	`workflow.transaction`, `ui_common.mutate()`	All-or-nothing; commit-time hard-rule guard
Domain	`workflow.py`, `optimizer.py`, `baseline.py`, `staffing.py`, `recovery.py`, `risk.py`, `priority.py`, `planning.py`, `requests.py`, `hr.py`, `photos.py`	Pure functions where possible
Data	`db.py`, `store.py`, `refdata.py`, `dataflow.py`, `convert.py`, `schemas.py`, `comms.py`	Versioned saves; append-only audit

Layer	Modules	Rule
Operations	`health.py`, `backup.py`, `pgbackup.py`, `jobs.py`, `migrate_to_pg.py`, `dropfolder.py`, `logs.py`	CLI wrappers in `scripts/`

Workspace model. The roster, requests and incidents live in a versioned *workspace* snapshot (table `workspace`, slots `live` and `scenario`). Every save carries the version it started from; a save based on an older version is refused ("Someone else changed this workspace…"). Read-model tables `live_roster` and `plan_versions` mirror each save for reporting.

Live vs demo. The demo scenario is a separate workspace slot with its own run id (`sim-…`). The simulation clock runs only there. The live clock only moves time.

Reference data. Employees, contracts, rules, service levels, leave, volumes and situations are versioned records (`ref_records`) changed through change sets (`change_sets`) with preview, approval and effective dates.

3. Planning engine

1. **Forecast** (`staffing.forecast`): the same-weekday average of recent weeks per queue and 30-minute slot, with a low/mid/high band.
2. **Erlang C** (`staffing.staffing_chain`): productive heads that meet the SLA target (for example 80 % in 20 s) given AHT; ÷ (1 – shrinkage) gives the required scheduled heads.
3. **CP-SAT** (`optimizer.plan`): decision variables per agent × day × shift. Hard rules are constraints. The objective minimises weighted under-staffing (highest), over-staffing, overtime, unfairness, preference misses, non-primary-queue assignments and changes to the published roster.
4. **Independent checker** (`constraints.check`): re-tests every hard rule after every solve and at every commit.
5. **Recovery** (`recovery.py`): generate (redeploy, swap, open shift, extension/overtime) → filter (hard rules, freeze windows, started shifts) → simulate → rank (cost + residual risk + blast radius) → local re-solve.

4. Data model

Entity relationship diagram

Logical relationships between the stored entities. Employees are versioned reference records (`ref_records`, domain `employees`) and the agent table inside the workspace; most HR tables refer to them by `employee_id`. Requests, incidents and the roster live inside the versioned workspace snapshot (`workspace`).

Diagram: see the Mermaid source in documentation/supporting (renders on GitHub).

Tables

Table	Purpose	Module
`accounts`	Sign-in accounts, contact, security state, theme	accounts.py
`audit_log`	Append-only, hash-chained audit trail	comms.py
`case_events`	Governance case history	comms.py
`cases`	Governance review cases	comms.py
`change_sets`	Data changes: submit, approve, apply	refdata.py / dataflow.py

Table	Purpose	Module
`emails`	Email attempts (masked recipient, status)	mailer.py
`employee_photos`	Profile photos (resized, metadata stripped)	photos.py
`hr_absence_flags`	Confidential absence-pattern flags	hr.py
`hr_attendance_corrections`	Attendance correction requests	hr.py
`hr_case_access`	Case access log	hr.py
`hr_case_events`	Restricted case entries	hr.py
`hr_cases`	Restricted HR cases	hr.py
`hr_leave_balances`	Opening leave balances	hr.py
`hr_leave_transactions`	Leave ledger	hr.py
`hr_overtime_tasks`	Overtime approval tasks	hr.py
`hr_policy`	Versioned HR policy settings	hr.py
`hr_review_tasks`	Leave review tasks	hr.py
`hr_separations`	Separation records	hr.py
`import_mappings`	Remembered column mappings for imports	refdata.py
`live_roster`	Read-model rows of the live roster	store.py
`messages`	Conversation threads	comms.py
`notices`	In-app notifications	comms.py
`plan_versions`	Read-model rows of plan versions	store.py
`ref_records`	Versioned reference data (employees, contracts, rules, leave, volumes, situations)	refdata.py
`reminders`	Attendance reminders / check-in windows	attendance.py
`scenario_workspace`		
`schedule_changes`		
`schema_migrations`	Applied migrations	db.py
`whatif_runs`	Saved what-if scenarios	whatif_store.py
`workspace`	Versioned live and demo workspace snapshots	store.py

Column-level detail: `data-dictionary.xlsx` (sheet Columns) and `application/database-migrations/`.

Column-level definitions: `supporting/data/data-dictionary.xlsx`. Schema DDL: `application/database-migrations/`. Lineage: `supporting/data/data-lineage-and-transformations.xlsx`.

5. Metric definitions

Metric	Definition	Basis
Required staffing	Erlang C productive heads ÷ (1 – shrinkage), rounded up	Modelled
Shortage	max(required – scheduled, 0); people-hours = Σ ÷ 2	Modelled
Coverage %	Σ min(scheduled, required) ÷ Σ required	Modelled
Labour / overtime cost	paid hours × contract rate; overtime hours × rate × multiplier	Simulated rates
Time to cover	alert raised → fixed or closed	Simulated time

Metric	Definition	Basis
Absenteeism	no-shows and absences ÷ recorded shift starts (no-record shifts excluded)	Recorded events
Forecast accuracy	needs imported actual volumes	Unavailable without actuals
Primary-queue share	hours on primary queue ÷ hours worked	Simulated

6. AI

There is no trained ML. AI (Anthropic API) is optional and off without a key. Rules run first; AI is used only when the rules find nothing: free-text request parsing (`assistant.FAST_MODEL`), column matching for imports (only with `SHIFTSENSE_AI_CONVERT`, non-personal by default) and answering open questions (`assistant.MODEL`). Explanations of fixes and priorities are deterministic. **The AI never makes a decision.**